SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-010: Student Profile Image Upload for the Admin Student Management

Field	Details
Change Request ID	CR-010
Change Request Title	Student Profile Image Upload for the Admin Student Management
Student Name	C. Lavasujan
Student ID	24ICT06
Implementation Status	Completed
Date	13 October 2026

1. Change Request Summary

1.1 Business Requirement

Currently, administrators can manage student information such as registration number, name, email address, and phone number. However, the system does not provide an option to upload or display a student's profile image. When managing a large number of students, a photographic profile helps administrators identify students more easily and improves the overall usability of the student management system. Therefore, the system was enhanced to allow administrators to upload a profile image when adding a new student, supporting standard image formats including PNG, JPG, and JPEG, and displaying the uploaded image across the student management interfaces.

1.2 Existing Behaviour

Before implementation:

- The administrator can access the student management section.
- The administrator can add a new student using the existing Add Student form.
- Existing student information such as name, registration number, email, and phone number can be entered.
- The Add Student form does not contain a profile image upload field.
- Students are displayed in the student list without profile images.
- No image file validation or upload handling is available.
- Student records are stored without a profile image reference in the database.
- Existing student CRUD operations use the existing student_id.

1.3 Requested Change

A profile image upload facility was requested for the administrator's Add Student form. The implementation must:

- Provide a profile image upload field in the Add Student form.
- Allow administrators to select a student image from their computer.
- Support PNG, JPG, and JPEG image formats.
- Reject unsupported and unsafe file formats (.exe, .js, .php, .html, .pdf, .txt).
- Validate the selected file on both client and server sides before uploading.
- Display a suitable validation message when an unsupported file is selected.
- Store the uploaded image securely on the server with sanitized unique filenames.
- Store the image path/reference with the corresponding student record in the database.
- Display the uploaded student image in the student list and student portal.
- Preserve existing Add Student, Edit, and Delete functionality.
- Ensure the uploaded image is associated with the correct student_id.
- Maintain existing administrator authentication and authorization controls.
- Provide a suitable default/placeholder image when no profile image is available.

2. Implementation

2.1 Implementation Summary

CR-010 introduces end-to-end profile image upload support across the frontend and backend. On the backend, Multer middleware was integrated with disk storage at backend/uploads/students/, enforcing strict extension and MIME-type validation for PNG, JPG, and JPEG formats, alongside a 5MB size limit. The database schema was upgraded by adding a profile_photo column to the students table to store the relative file path. In Express, static asset serving was enabled on /uploads. On the frontend, StudentFormModal.jsx was updated with an accessible file upload input, client-side format checks, instant thumbnail previews, and multipart/form-data submission. DashboardPage.jsx was enhanced with an Image table column displaying thumbnail photos or stylized initial avatars for students without images. Existing CRUD functionality, authorization controls, and student ID integrity were fully preserved.

2.2 Affected Files

File	Change	Purpose
database/schema.sql	Modified	Added profile_photo VARCHAR(255) column to students table with migration statement.
backend/package.json	Modified	Added multer dependency for multipart/form-data upload handling.
backend/src/middleware/uploadMiddleware.js	Created	Configured Multer disk storage, secure random filenames, format whitelisting, and error responses.
backend/src/app.js	Modified	Registered express.static middleware to serve /uploads directory statically.
backend/src/routes/studentRoutes.js	Modified	Bound uploadStudentPhoto middleware to POST /api/students route.
backend/src/controllers/studentController.js	Modified	Stored profile_photo path, cleaned up orphaned uploads on failure, and deleted files on student removal.
backend/src/models/studentModel.js	Modified	Updated createStudent insert query and included profile_photo in student select queries.
frontend/src/services/api.js	Modified	Added FormData support in apiRequest without overriding multipart boundary headers.
frontend/src/components/StudentFormModal.jsx	Modified	Added profile image file input, format validation, preview thumbnail, and multipart submission.
frontend/src/pages/DashboardPage.jsx	Modified	Added Image table header, rendered student profile thumbnails and fallback placeholder avatars.
frontend/src/pages/StudentDashboardPage.jsx	Modified	Rendered uploaded student profile photo in student portal profile card.
frontend/src/styles/dashboard.css	Modified	Added styles for table image thumbnails, fallback avatar circles, file inputs, and dark mode.
frontend/src/styles/studentDashboard.css	Modified	Added .profile-avatar-img styling for student profile view.

Additional files: backend/uploads/students/ (Directory used for local profile photo storage).

2.3 Technical Changes

Upload Handling & Storage Configuration

Multer disk storage was configured in uploadMiddleware.js:

- Destination directory: backend/uploads/students/, auto-created if missing.
- Filename generation: student_<timestamp>_<random>.<ext> to prevent collisions and directory traversal.
- Format whitelist: Strict verification of extension (.png, .jpg, .jpeg) and MIME type (image/png, image/jpeg, image/pjpeg).
- File size enforcement: 5MB threshold with LIMIT_FILE_SIZE error handling.
- Invalid formats rejected with HTTP 400: 'Only PNG, JPG and JPEG images are allowed.'

Database Schema & Persistence

- Column added: profile_photo VARCHAR(255) NULL DEFAULT NULL in students table.
- Stores relative reference path (e.g., /uploads/students/student_1791874642702_488233575.png) rather than binary BLOBs.
- Included in all queries: getAllStudents, getStudentById, and getStudentByUserId.

Frontend Form & Multi-part Submission

- Added file input with accept=".png,.jpg,.jpeg,image/png,image/jpeg" in Add Student modal.
- Live client validation displays instant error message when unsupported files are selected.
- Object URL preview allows administrator to inspect image prior to submission.
- Multipart FormData payload seamlessly handles student text attributes and binary file data.

Display & Fallback Placeholder

- Table displays 40x40px rounded thumbnail with subtle border and box-shadow.
- Students without photos display circular gradient placeholder with student initial.
- Student portal renders 60x60px profile avatar in StudentDashboardPage.jsx.

2.4 Implementation Logic

1. Administrator authenticates and navigates to the Admin Dashboard.
2. Administrator clicks '+ Add Student' button to open StudentFormModal.
3. Administrator enters registration number, name, email, phone, and password.
4. Administrator clicks Choose File under Profile Image and selects an image.
5. Frontend inspects extension and MIME type. If invalid (e.g., .pdf, .exe, .txt), file is rejected with 'Only PNG, JPG and JPEG images are allowed.' error banner.
6. If valid (PNG/JPG/JPEG), file is accepted and an instant image preview is displayed.
7. Administrator clicks Create Student. FormData payload transmits all fields + profilePhoto.
8. Server-side Multer middleware validates file format, generates unique safe filename, and writes file to backend/uploads/students/.
9. studentController hashes password, creates user record, and inserts student record linking profile_photo path to the generated student_id.
10. Student table automatically updates and renders new student record displaying uploaded profile image.

2.5 Error and Data Handling

Comprehensive error handling and storage lifecycle management were implemented:

- Invalid file formats (.pdf, .exe, .js, .txt) return HTTP 400 with 'Only PNG, JPG and JPEG images are allowed.'
- Orphaned file cleanup: If validation fails or database insertion errors occur after upload, the file is automatically purged from disk.
- Student deletion cleanup: Deleting a student via DELETE /api/students/:id automatically removes their associated image file from disk.
- Existing duplicate checks (username, email) and format validations remain strictly intact.

3. Testing & Verification

3.1 Testing Approach

The implementation was verified through automated test suites and end-to-end integration testing covering backend API routes, upload security, static file serving, database persistence, and UI rendering.

Testing Types:

- Functional testing (image upload, display, CRUD operations)
- Format validation testing (PNG, JPG, JPEG acceptance)
- Negative testing (rejection of PDF, EXE, TXT, and unsafe files)
- Security & authorization testing (admin vs. student permissions)
- Regression testing (ensuring existing CRUD, search, and login remain intact)
- UI & placeholder testing (verifying fallback avatar rendering)

Environment: Development (Node.js Express + MySQL + Vite React)
Browser: [Chrome / Edge / Firefox]
Test Role: Administrator & Student

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Add student without image	Student added successfully; profile_photo stored as null	Status 201; student created with null photo	Pass
TC-02	Upload PNG image (student.png)	PNG accepted, stored in /uploads/students/, path saved	Status 201; stored as .png, path linked	Pass
TC-03	Upload JPG image (student.jpg)	JPG accepted, stored in /uploads/students/, path saved	Status 201; stored as .jpg, path linked	Pass
TC-04	Upload JPEG image (student.jpeg)	JPEG accepted, stored in /uploads/students/, path saved	Status 201; stored as .jpeg, path linked	Pass

Test ID	Test Case	Expected Result	Actual Result	Status
TC-05	Upload PDF file (student.pdf)	File rejected with 400 error: 'Only PNG, JPG and JPEG images are allowed.'	Status 400; clear rejection message returned	Pass
TC-06	Upload EXE file (student.exe)	File rejected with 400 error: 'Only PNG, JPG and JPEG images are allowed.'	Status 400; unsafe file rejected	Pass
TC-07	Upload unsupported file (document.txt)	File rejected with 400 error: 'Only PNG, JPG and JPEG images are allowed.'	Status 400; rejected with validation message	Pass
TC-08	Add student with valid image	Student and image saved; student_id linked to photo path	Status 201; student created and linked	Pass
TC-09	Display uploaded image	Correct image served via /uploads/ and rendered in table	Status 200; image served and displayed	Pass
TC-10	Student without image displays placeholder	Default stylized avatar with student initial displayed	Initial avatar rendered in table & portal	Pass
TC-11	Edit student after image upload	Student details updated without altering profile photo	Status 200; details updated, photo preserved	Pass
TC-12	Delete student after image upload	Student deleted from DB and photo removed from disk	Status 200; record deleted, file unlinked	Pass
TC-13	Unauthorized upload attempt	Non-admin student receives 403 Forbidden on upload route	Status 403; access denied to unauthorized user	Pass

3.3 Test Summary

Test Result	Count
Total Test Cases	13
Passed	13
Failed	0
Pass Rate	100%

Overall Test Result: Pass

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	The Add Student form displays a Profile Image upload field.	UI verification in modal (TC-01, TC-08)	Met
AC-002	Administrators can select a PNG image.	Format acceptance test (TC-02)	Met
AC-003	Administrators can select a JPG image.	Format acceptance test (TC-03)	Met
AC-004	Administrators can select a JPEG image.	Format acceptance test (TC-04)	Met

ID	Acceptance Criterion	Verification	Status
AC-005	Unsupported file formats are rejected.	Validation tests (TC-05, TC-06, TC-07)	Met
AC-006	A clear error message is displayed when an invalid file is selected.	Message verification (TC-05, TC-06, TC-07)	Met
AC-007	A valid profile image is successfully uploaded when a student is created.	Upload workflow test (TC-08)	Met
AC-008	The uploaded image is associated with the correct student.	Database integrity test (TC-08, TC-09)	Met
AC-009	The uploaded image is displayed correctly in the student list or student details page.	Image rendering inspection (TC-09)	Met
AC-010	Students without an image display a default/placeholder image.	Placeholder display check (TC-10)	Met
AC-011	Existing Edit and Delete operations continue to affect the correct student.	CRUD regression tests (TC-11, TC-12)	Met
AC-012	Existing administrator authentication and authorization continue to work.	Security test (TC-13)	Met
AC-013	No unsupported or unsafe file type can be uploaded through the profile image field.	Server-side whitelist test (TC-06)	Met

5. Evidence

A maximum of two relevant screenshots should be attached.

Evidence 01 – Add Student Form with Profile Image Upload Field & Format Validation

Related: AC-001, AC-002, AC-003, AC-004, AC-005, AC-006

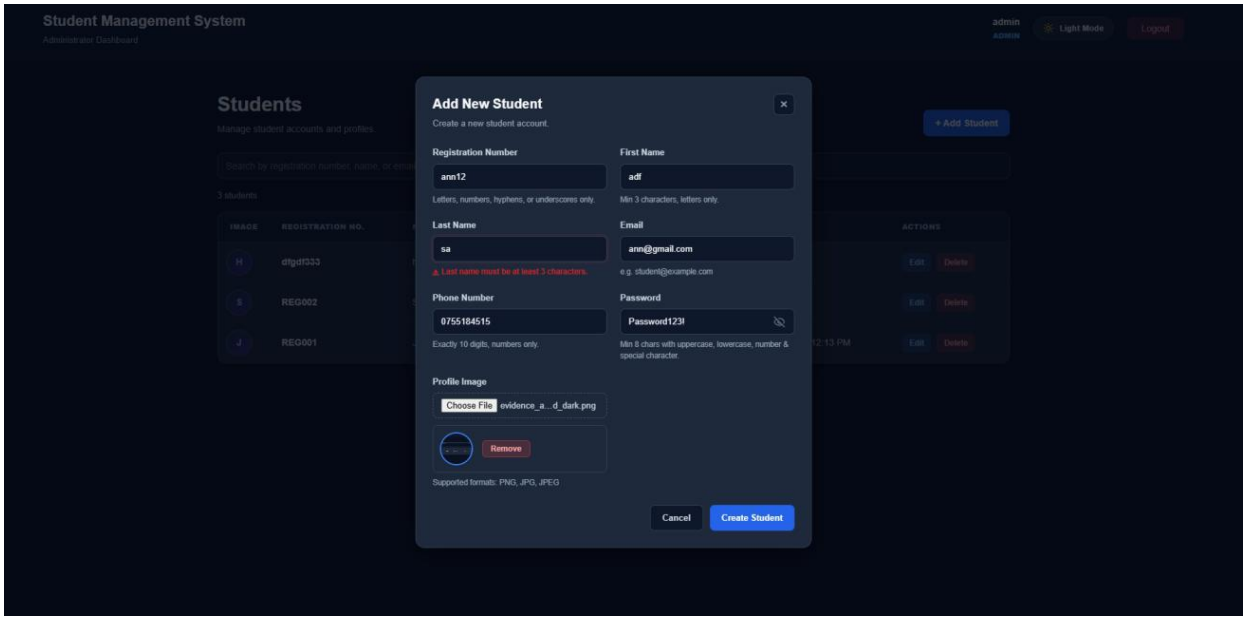


Figure 1: Add Student form displaying Profile Image upload field, file format hint, and format validation.

Evidence 02 – Student List Displaying Uploaded Profile Images and Placeholders

Related: AC-007, AC-008, AC-009, AC-010, AC-011, AC-012, AC-013

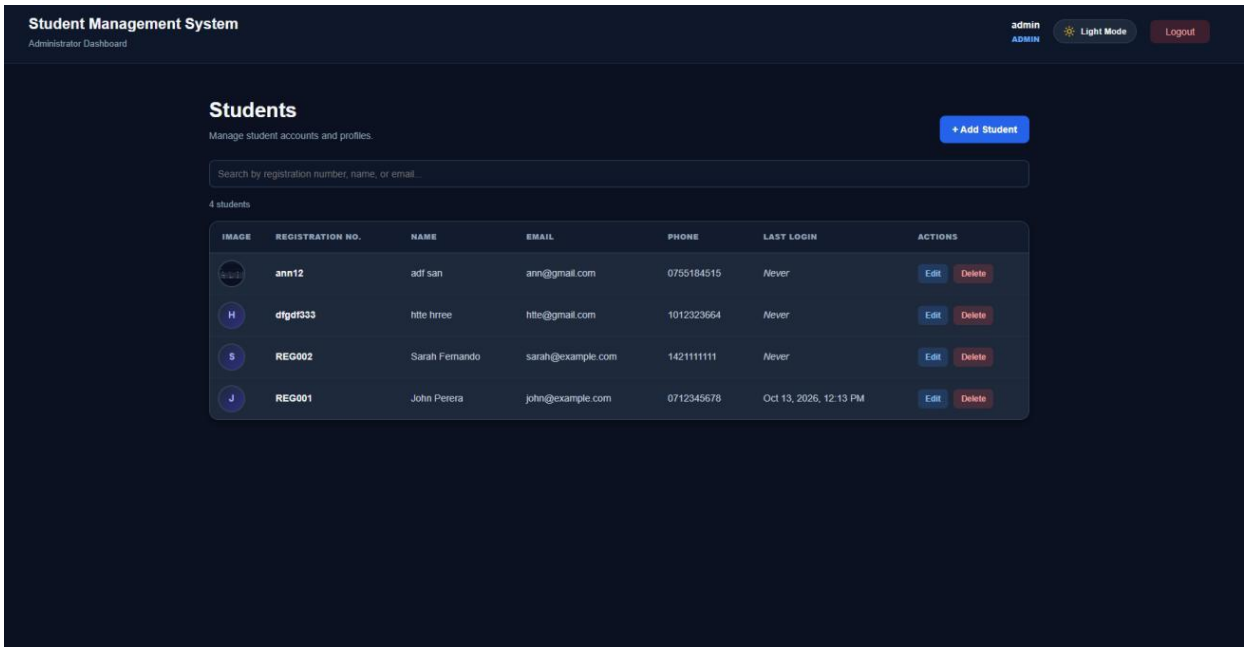


Figure 2: Admin Dashboard student list table displaying profile photo thumbnail alongside student records and initials fallback placeholder.

6. Final Result

Item	Result
Requirements	Met
Implementation	Completed
Testing	Passed
Acceptance Criteria	All Met
Existing CRUD / Auth	Verified
Final Status	Accepted

6.1 Final Summary

CR-010 was successfully implemented as a full-stack enhancement introducing student profile image upload functionality to the Student Management System. Administrators can upload

student profile images in PNG, JPG, or JPEG format during student creation. The system validates selected files on both client and server sides, stores files securely on the server with sanitized unique filenames, associates the image reference with the student record, and displays the image in the student management table and student portal. For students without images, a styled fallback placeholder avatar is displayed. Existing authentication, authorization, and CRUD operations continue to work without regression. The change was thoroughly verified against all defined functional requirements and acceptance criteria with a 100% test pass rate.

6.2 Issues / Limitations

[State any remaining issues or limitations.]

If none:

No significant issues identified.

Student Name: C. Lavasujan

Student ID: 24ICT06

Date: 13 October 2026

Signature: _____

~ END OF REPORT ~